

Object-Oriented Programming

TCS NQT - Core CS Subjects

TCS NQT Preparation Material
Generated on: April 05, 2026

Section 1: Classes & Objects

Constructors, destructors, this pointer, static members, and basic class concepts.

Q1. [Easy]

What is a class in OOP?

- A) An instance of an object
- B) A blueprint for creating objects
- C) A type of function
- D) A variable type

Answer: B) A blueprint for creating objects

Explanation: A class defines the structure (attributes) and behavior (methods) that objects of that type will have.

Q2. [Easy]

Which constructor is called when no arguments are passed during object creation?

- A) Parameterized constructor
- B) Copy constructor
- C) Default constructor
- D) Conversion constructor

Answer: C) Default constructor

Explanation: A default constructor takes no arguments. If no constructor is defined, the compiler provides one.

Q3. [Easy]

What is the purpose of a destructor?

- A) To create objects
- B) To initialize objects
- C) To release resources when an object is destroyed
- D) To copy objects

Answer: C) To release resources when an object is destroyed

Explanation: Destructors are called when an object goes out of scope or is deleted, freeing allocated resources.

Q4. [Easy]

In C++, what does the 'this' pointer refer to?

- A)** The class itself
- B)** The current object
- C)** The parent class
- D)** The base class

Answer: B) The current object

Explanation: The 'this' pointer is an implicit pointer available in non-static member functions, pointing to the calling object.

Q5. [Medium]

A static member variable in a class is:

- A)** Unique to each object
- B)** Shared among all objects of the class
- C)** Only accessible in constructors
- D)** Automatically initialized to null

Answer: B) Shared among all objects of the class

Explanation: Static members belong to the class, not individual objects. All objects share the same copy.

Q6. [Medium]

Can a static member function access non-static data members?

- A)** Yes, always
- B)** No, it cannot
- C)** Only if the object is passed as parameter
- D)** Only in the constructor

Answer: C) Only if the object is passed as parameter

Explanation: Static functions have no 'this' pointer, so they cannot directly access non-static members. An object reference must be passed.

Q7. [Medium]

What is the output when a copy constructor is invoked?

- A) A new object identical to the original
- B) A reference to the original object
- C) A null object
- D) A shallow copy only

Answer: A) A new object identical to the original

Explanation: A copy constructor creates a new object as a copy of an existing object. By default it performs a shallow copy.

Q8. [Easy]

In C++, which keyword is used to define a class?

- A) struct
- B) class
- C) object
- D) type

Answer: B) class

Explanation: The 'class' keyword defines a class. 'struct' can also define a class in C++, but members are public by default.

Q9. [Medium]

What is the difference between a struct and a class in C++?

- A) No difference
- B) Struct members are public by default, class members are private
- C) Struct cannot have methods
- D) Class cannot have data members

Answer: B) Struct members are public by default, class members are private

Explanation: The only difference in C++ is default access: struct = public, class = private.

Q10. [Medium]

How many times is the constructor called when creating an array of 5 objects?

- A)** 1
- B)** 5
- C)** 0
- D)** 10

Answer: B) 5

Explanation: The default constructor is called once for each element in the array, so 5 times for an array of 5.

Section 2: Inheritance

Single, multiple, multilevel, hierarchical inheritance, diamond problem, and virtual inheritance.

Q11. [Easy]

Inheritance in OOP represents which relationship?

- A) Has-a
- B) Is-a
- C) Uses-a
- D) Part-of

Answer: B) Is-a

Explanation: Inheritance models an 'is-a' relationship (e.g., Dog is-a Animal). Composition models 'has-a'.

Q12. [Medium]

In multilevel inheritance, class C inherits from B which inherits from A. What can C access?

- A) Only B members
- B) Only A members
- C) Both A and B public/protected members
- D) Neither A nor B members

Answer: C) Both A and B public/protected members

Explanation: In multilevel inheritance, the derived class can access public and protected members of all ancestor classes.

Q13. [Medium]

The diamond problem occurs in:

- A) Single inheritance
- B) Multilevel inheritance
- C) Multiple inheritance
- D) Hierarchical inheritance

Answer: C) Multiple inheritance

Explanation: The diamond problem occurs when a class inherits from two classes that share a common base class, creating ambiguity.

Q14. [Medium]

How is the diamond problem resolved in C++?

- A)** Using abstract classes
- B)** Using virtual inheritance
- C)** Using interfaces
- D)** Using static members

Answer: B) Using virtual inheritance

Explanation: Virtual inheritance ensures only one copy of the base class exists, resolving the diamond problem ambiguity.

Q15. [Medium]

Which type of inheritance is NOT supported in Java?

- A)** Single
- B)** Multilevel
- C)** Multiple (class)
- D)** Hierarchical

Answer: C) Multiple (class)

Explanation: Java does not support multiple inheritance of classes to avoid the diamond problem. It supports multiple inheritance through interfaces.

Q16. [Hard]

In private inheritance (C++), public members of the base class become:

- A)** Public in derived class
- B)** Protected in derived class
- C)** Private in derived class
- D)** Inaccessible

Answer: C) Private in derived class

Explanation: In private inheritance, all public and protected members of the base class become private in the derived class.

Q17. [Easy]

What is hierarchical inheritance?

- A) One class inherits from multiple classes
- B) Multiple classes inherit from a single base class
- C) Chain of inheritance
- D) Inheritance with interfaces

Answer: B) Multiple classes inherit from a single base class

Explanation: Hierarchical inheritance: multiple derived classes inherit from one base class (e.g., Dog, Cat, Bird from Animal).

Q18. [Medium]

In C++, the order of constructor calls in inheritance is:

- A) Derived first, then base
- B) Base first, then derived
- C) Random order
- D) Only derived constructor

Answer: B) Base first, then derived

Explanation: Constructors are called from base to derived. Destructors are called in reverse order (derived first).

Q19. [Easy]

Which keyword is used in Java to inherit a class?

- A) inherits
- B) extends
- C) implements
- D) derives

Answer: B) extends

Explanation: Java uses 'extends' for class inheritance and 'implements' for interface implementation.

Q20. [Medium]

Can a constructor be inherited?

- A) Yes, always
- B) No, constructors are not inherited
- C) Only default constructors
- D) Only in Java

Answer: B) No, constructors are not inherited

Explanation: Constructors are not inherited. Each class must define its own. The base constructor is called via initialization list or `super()`.

Q21. [Easy]

What is the 'super' keyword used for in Java?

- A) Access static members
- B) Call parent class constructor/methods
- C) Create new objects
- D) Define interfaces

Answer: B) Call parent class constructor/methods

Explanation: The 'super' keyword in Java refers to the parent class. `super()` calls the parent constructor, `super.method()` calls parent methods.

Q22. [Hard]

In protected inheritance (C++), public members of base become:

- A) Public
- B) Protected
- C) Private
- D) Inaccessible

Answer: B) Protected

Explanation: In protected inheritance, public members of the base class become protected in the derived class.

Section 3: Polymorphism

Compile-time polymorphism (overloading), runtime polymorphism (overriding), virtual functions, and vtable.

Q23. [Easy]

Compile-time polymorphism is achieved through:

- A) Virtual functions
- B) Function overloading and operator overloading
- C) Inheritance
- D) Abstract classes

Answer: B) Function overloading and operator overloading

Explanation: Compile-time (static) polymorphism is resolved at compile time via function overloading, operator overloading, and templates.

Q24. [Easy]

Runtime polymorphism is achieved through:

- A) Function overloading
- B) Operator overloading
- C) Virtual functions and function overriding
- D) Templates

Answer: C) Virtual functions and function overriding

Explanation: Runtime (dynamic) polymorphism uses virtual functions and is resolved at runtime through vtable lookup.

Q25. [Medium]

What is a virtual function in C++?

- A) A function with no body
- B) A function that can be overridden in derived class with dynamic dispatch
- C) A static function
- D) A template function

Answer: B) A function that can be overridden in derived class with dynamic dispatch

Explanation: Virtual functions enable dynamic dispatch - the actual function called depends on the runtime type of the object.

Q26. [Medium]

What is a vtable?

- A) A table of variables
- B) A table of function pointers for virtual functions
- C) A vector table
- D) A validation table

Answer: B) A table of function pointers for virtual functions

Explanation: The vtable (virtual table) is a lookup table of function pointers used to resolve virtual function calls at runtime.

Q27. [Medium]

Can constructors be virtual in C++?

- A) Yes
- B) No
- C) Only default constructors
- D) Only copy constructors

Answer: B) No

Explanation: Constructors cannot be virtual because the vtable is not set up until the constructor completes. Destructors can be virtual.

Q28. [Hard]

Why should destructors be declared virtual in base classes?

- A) For better performance
- B) To ensure proper cleanup of derived class objects when deleted via base pointer
- C) It is mandatory
- D) To prevent inheritance

Answer: B) To ensure proper cleanup of derived class objects when deleted via base pointer

Explanation: Without a virtual destructor, deleting a derived object through a base pointer causes undefined behavior (only base destructor runs).

Q29. [Easy]

What is function overloading?

- A)** Multiple functions with the same name but different parameters
- B)** Redefining a base class function in derived class
- C)** A function calling itself
- D)** A function with default arguments

Answer: A) Multiple functions with the same name but different parameters

Explanation: Function overloading allows multiple functions with the same name but different parameter lists (number, type, or order).

Q30. [Easy]

What is function overriding?

- A)** Same function name with different parameters
- B)** Redefining a base class virtual function in derived class with same signature
- C)** Calling parent function
- D)** Operator overloading

Answer: B) Redefining a base class virtual function in derived class with same signature

Explanation: Overriding redefines a virtual base function in the derived class with the exact same signature for runtime polymorphism.

Q31. [Medium]

Which operator cannot be overloaded in C++?

- A)** +
- B)** -
- C)** ::
- D)** ==

Answer: C) ::

Explanation: The scope resolution (::), member access (.), sizeof, and ternary (?:) operators cannot be overloaded in C++.

Q32. [Easy]

In Java, what annotation is used to indicate method overriding?

- A) @Override
- B) @Overload
- C) @Virtual
- D) @Super

Answer: A) @Override

Explanation: @Override annotation helps the compiler verify that the method actually overrides a superclass method.

Q33. [Medium]

A pure virtual function in C++ is declared with:

- A) virtual keyword only
- B) = 0 at the end
- C) abstract keyword
- D) override keyword

Answer: B) = 0 at the end

Explanation: A pure virtual function is declared as: virtual void func() = 0; It makes the class abstract.

Q34. [Medium]

What is early binding vs late binding?

- A) Early = runtime, Late = compile-time
- B) Early = compile-time, Late = runtime
- C) Both are compile-time
- D) Both are runtime

Answer: B) Early = compile-time, Late = runtime

Explanation: Early binding resolves function calls at compile time (overloading). Late binding resolves at runtime (virtual functions).

Section 4: Encapsulation & Abstraction

Access modifiers, abstract classes, interfaces, and data hiding.

Q35. [Easy]

Encapsulation is:

- A) Inheritance of properties
- B) Bundling data and methods that operate on it, restricting direct access
- C) Function overloading
- D) Multiple inheritance

Answer: B) Bundling data and methods that operate on it, restricting direct access

Explanation: Encapsulation wraps data and methods into a single unit (class) and controls access via access modifiers.

Q36. [Easy]

Which access modifier makes a member accessible only within the same class?

- A) public
- B) protected
- C) private
- D) default

Answer: C) private

Explanation: Private members are accessible only within the class itself. Protected allows access in derived classes too.

Q37. [Medium]

An abstract class in Java:

- A) Cannot have any methods
- B) Can have both abstract and concrete methods
- C) Must have only abstract methods
- D) Cannot be inherited

Answer: B) Can have both abstract and concrete methods

Explanation: Abstract classes can have abstract methods (no body) and concrete methods (with implementation). They cannot be instantiated.

Q38. [Medium]

What is the difference between an abstract class and an interface in Java?

- A)** No difference
- B)** Abstract class can have state and constructors; interface cannot (pre-Java 8)
- C)** Interface supports multiple inheritance; abstract class does not
- D)** Both B and C

Answer: D) Both B and C

Explanation: Abstract classes can have instance variables and constructors. A class can implement multiple interfaces but extend only one class.

Q39. [Easy]

Abstraction focuses on:

- A)** Hiding implementation details and showing only essential features
- B)** Inheriting properties
- C)** Creating multiple objects
- D)** Data binding

Answer: A) Hiding implementation details and showing only essential features

Explanation: Abstraction hides complexity by providing a simplified interface. Users see what an object does, not how.

Q40. [Medium]

In Java, can an interface have a constructor?

- A)** Yes
- B)** No
- C)** Only default constructor
- D)** Only parameterized constructor

Answer: B) No

Explanation: Interfaces cannot have constructors because they cannot be instantiated and have no instance state to initialize.

Q41. [Medium]

Since Java 8, interfaces can have:

- A)** Only abstract methods
- B)** Default and static methods
- C)** Constructors
- D)** Instance variables

Answer: B) Default and static methods

Explanation: Java 8 introduced default methods (with implementation) and static methods in interfaces.

Q42. [Easy]

Which is the best example of encapsulation?

- A)** Using inheritance
- B)** Private data members with public getters/setters
- C)** Using virtual functions
- D)** Using templates

Answer: B) Private data members with public getters/setters

Explanation: Making data private and providing controlled access through public getters/setters is the classic encapsulation pattern.

Section 5: Design Patterns

Singleton, Factory, Observer, Strategy, MVC and other common patterns.

Q43. [Easy]

The Singleton pattern ensures:

- A) Multiple instances of a class
- B) Only one instance of a class exists
- C) Inheritance of classes
- D) Polymorphic behavior

Answer: B) Only one instance of a class exists

Explanation: Singleton restricts a class to a single instance and provides a global access point to it.

Q44. [Easy]

Which design pattern creates objects without specifying the exact class?

- A) Singleton
- B) Factory
- C) Observer
- D) Strategy

Answer: B) Factory

Explanation: The Factory pattern uses a factory method to create objects, allowing subclasses to determine the actual type.

Q45. [Medium]

The Observer pattern implements:

- A) One-to-one relationship
- B) One-to-many dependency (publish-subscribe)
- C) Many-to-many relationship
- D) No relationship

Answer: B) One-to-many dependency (publish-subscribe)

Explanation: Observer defines a one-to-many dependency where when one object (subject) changes state, all dependents (observers) are notified.

Q46. [Medium]

The Strategy pattern is used to:

- A)** Create a single instance
- B)** Define a family of algorithms and make them interchangeable
- C)** Observe state changes
- D)** Build complex objects

Answer: B) Define a family of algorithms and make them interchangeable

Explanation: Strategy encapsulates algorithms into separate classes, allowing the algorithm to be selected at runtime.

Q47. [Easy]

In MVC architecture, which component handles user input?

- A)** Model
- B)** View
- C)** Controller
- D)** All of the above

Answer: C) Controller

Explanation: Controller handles user input, updates the Model, and selects the View. Model manages data, View displays it.

Q48. [Medium]

The Decorator pattern is used to:

- A)** Create objects
- B)** Add responsibilities to objects dynamically
- C)** Restrict instantiation
- D)** Sort objects

Answer: B) Add responsibilities to objects dynamically

Explanation: Decorator wraps an object to add new behavior without modifying the original class. It is an alternative to subclassing.

Q49. [Medium]

The Adapter pattern is used to:

- A)** Create a single instance
- B)** Make incompatible interfaces work together
- C)** Observe changes
- D)** Build objects step by step

Answer: B) Make incompatible interfaces work together

Explanation: The Adapter pattern converts the interface of a class into another interface that clients expect.

Q50. [Medium]

Which design pattern separates object construction from its representation?

- A)** Factory
- B)** Builder
- C)** Prototype
- D)** Singleton

Answer: B) Builder

Explanation: The Builder pattern constructs complex objects step by step, separating construction from representation.

Q51. [Medium]

The Prototype pattern creates new objects by:

- A)** Using a factory
- B)** Cloning existing objects
- C)** Direct instantiation
- D)** Lazy initialization

Answer: B) Cloning existing objects

Explanation: Prototype creates new objects by copying (cloning) an existing object, avoiding expensive creation from scratch.

Q52. [Easy]

Design patterns are categorized into:

- A)** Structural, Behavioral, Functional
- B)** Creational, Structural, Behavioral
- C)** Abstract, Concrete, Virtual
- D)** Static, Dynamic, Hybrid

Answer: B) Creational, Structural, Behavioral

Explanation: Gang of Four categorized 23 patterns: Creational (object creation), Structural (composition), Behavioral (interaction).

Section 6: C++ Specific OOP

Templates, STL, smart pointers, move semantics, RAII, and other C++ specific features.

Q53. [Easy]

What is a template in C++?

- A) A class that cannot be instantiated
- B) A mechanism for generic programming
- C) A type of inheritance
- D) An interface

Answer: B) A mechanism for generic programming

Explanation: Templates allow writing generic code that works with any data type, resolved at compile time.

Q54. [Medium]

Which STL container provides $O(1)$ average time for insertion and lookup?

- A) vector
- B) list
- C) unordered_map
- D) map

Answer: C) unordered_map

Explanation: unordered_map uses hash table for $O(1)$ average time. map uses balanced BST for $O(\log n)$.

Q55. [Medium]

What is RAII in C++?

- A) Random Access Indexed Arrays
- B) Resource Acquisition Is Initialization
- C) Runtime Abstraction Interface Implementation
- D) Reference Allocation In Inheritance

Answer: B) Resource Acquisition Is Initialization

Explanation: RAII ties resource lifecycle to object lifetime. Resources are acquired in constructor and released in destructor.

Q56. [Medium]

Which smart pointer allows exactly one owner?

- A) `shared_ptr`
- B) `unique_ptr`
- C) `weak_ptr`
- D) `auto_ptr`

Answer: B) `unique_ptr`

Explanation: `unique_ptr` has exclusive ownership. It cannot be copied, only moved. `shared_ptr` allows multiple owners.

Q57. [Hard]

What does `std::move()` do?

- A) Moves memory physically
- B) Casts an lvalue to an rvalue reference enabling move semantics
- C) Deletes the object
- D) Copies the object

Answer: B) Casts an lvalue to an rvalue reference enabling move semantics

Explanation: `std::move()` is a cast that enables move semantics by converting an lvalue to an rvalue reference, allowing resource transfer.

Q58. [Hard]

What is the difference between `shared_ptr` and `weak_ptr`?

- A) No difference
- B) `weak_ptr` does not increase reference count and can break circular references
- C) `shared_ptr` is faster
- D) `weak_ptr` owns the resource

Answer: B) `weak_ptr` does not increase reference count and can break circular references

Explanation: `weak_ptr` observes a `shared_ptr` without affecting the reference count, preventing circular reference memory leaks.

Q59. [Easy]

The STL vector stores elements in:

- A)** Linked list
- B)** Contiguous memory
- C)** Hash table
- D)** Binary tree

Answer: B) Contiguous memory

Explanation: `std::vector` stores elements in contiguous memory, providing $O(1)$ random access and cache-friendly iteration.

Q60. [Hard]

What is the Rule of Three in C++?

- A)** A class should have at most 3 members
- B)** If a class needs a custom destructor, copy constructor, or copy assignment, it likely needs all three
- C)** Use 3 levels of inheritance
- D)** Define 3 constructors

Answer: B) If a class needs a custom destructor, copy constructor, or copy assignment, it likely needs all three

Explanation: The Rule of Three states that if any one of these three special member functions is needed, all three should be defined.

Q61. [Hard]

What is the Rule of Five in C++11?

- A)** Five design patterns
- B)** Rule of Three plus move constructor and move assignment operator
- C)** Five levels of access
- D)** Five types of inheritance

Answer: B) Rule of Three plus move constructor and move assignment operator

Explanation: C++11 extends the Rule of Three with move constructor and move assignment operator for efficient resource transfer.

Q62. [Easy]

Which STL container is a FIFO data structure?

- A) stack
- B) queue
- C) vector
- D) set

Answer: B) queue

Explanation: `std::queue` is FIFO (First In, First Out). `std::stack` is LIFO (Last In, First Out).

Q63. [Medium]

What is the time complexity of `std::map` operations (insert, find, delete)?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Answer: B) $O(\log n)$

Explanation: `std::map` is implemented as a balanced BST (typically Red-Black Tree), so all operations are $O(\log n)$.

Q64. [Medium]

What keyword in C++ prevents a class from being inherited?

- A) static
- B) const
- C) final
- D) sealed

Answer: C) final

Explanation: The 'final' keyword (C++11) prevents a class from being inherited or a virtual function from being overridden.

Q65. [Hard]

What is the purpose of the 'explicit' keyword in C++?

- A) Make methods virtual
- B) Prevent implicit conversions via constructors
- C) Define pure virtual functions
- D) Enable multiple inheritance

Answer: B) Prevent implicit conversions via constructors

Explanation: 'explicit' on a constructor prevents the compiler from using it for implicit type conversions.

Q66. [Medium]

Lambda expressions in C++ are:

- A) Named functions
- B) Anonymous function objects
- C) Macros
- D) Templates

Answer: B) Anonymous function objects

Explanation: Lambdas are anonymous functions: `[capture](params){ body }`. They create a closure object.

Q67. [Easy]

What does the 'auto' keyword do in C++11?

- A) Creates automatic variables on the stack
- B) Deduces the type from the initializer
- C) Makes variables global
- D) Declares a pointer

Answer: B) Deduces the type from the initializer

Explanation: 'auto' lets the compiler deduce the variable type from its initializer: `auto x = 5; // int`

Section 7: Java / General OOP

Garbage collection, generics, exception handling, and Java-specific OOP concepts.

Q68. [Easy]

Java uses which type of memory management?

- A) Manual memory management
- B) Automatic garbage collection
- C) Reference counting only
- D) Stack-only allocation

Answer: B) Automatic garbage collection

Explanation: Java's garbage collector automatically reclaims memory from objects that are no longer referenced.

Q69. [Medium]

Which method is called by the garbage collector before destroying an object in Java?

- A) destroy()
- B) finalize()
- C) dispose()
- D) close()

Answer: B) finalize()

Explanation: The finalize() method is called before garbage collection. However, it is deprecated since Java 9.

Q70. [Medium]

What are Java generics used for?

- A) Dynamic typing
- B) Type-safe parameterized types
- C) Memory management
- D) Multi-threading

Answer: B) Type-safe parameterized types

Explanation: Generics provide compile-time type safety: List<String> ensures only strings are stored.

Q71. [Medium]

In Java exception handling, what is the difference between checked and unchecked exceptions?

- A)** No difference
- B)** Checked must be caught or declared; unchecked do not
- C)** Unchecked are more severe
- D)** Checked are runtime only

Answer: B) Checked must be caught or declared; unchecked do not

Explanation: Checked exceptions (IOException) must be handled. Unchecked (RuntimeException like NullPointerException) are optional to catch.

Q72. [Easy]

The 'final' keyword in Java can be applied to:

- A)** Variables only
- B)** Methods only
- C)** Classes only
- D)** Variables, methods, and classes

Answer: D) Variables, methods, and classes

Explanation: final variable = constant, final method = cannot override, final class = cannot inherit.

Q73. [Medium]

What is the output of: `String s1 = "hello"; String s2 = "hello"; System.out.println(s1 == s2);`

- A)** true
- B)** false
- C)** Compilation error
- D)** Runtime error

Answer: A) true

Explanation: Both string literals point to the same object in the String Pool, so == returns true for reference comparison.

Q74. [Medium]

What is the purpose of the 'try-with-resources' in Java?

- A)** Exception handling
- B)** Automatic resource management (auto-close)
- C)** Multi-threading
- D)** Garbage collection

Answer: B) Automatic resource management (auto-close)

Explanation: try-with-resources automatically closes resources implementing AutoCloseable when the try block exits.

Q75. [Easy]

In Java, all classes implicitly extend:

- A)** java.lang.Class
- B)** java.lang.Object
- C)** java.lang.Base
- D)** java.lang.Root

Answer: B) java.lang.Object

Explanation: Every class in Java implicitly extends java.lang.Object, which provides methods like equals(), hashCode(), toString().

Q76. [Hard]

What is method hiding in Java?

- A)** Making methods private
- B)** A static method in subclass with same signature as parent static method
- C)** Deleting methods
- D)** Method overloading

Answer: B) A static method in subclass with same signature as parent static method

Explanation: When a subclass defines a static method with the same signature as a parent static method, it hides (not overrides) it.

Q77. [Easy]

Which collection interface does NOT allow duplicate elements in Java?

- A) List
- B) Set
- C) Queue
- D) Deque

Answer: B) Set

Explanation: Set interface (HashSet, TreeSet) does not allow duplicates. List allows duplicates.

Q78. [Medium]

What is the diamond operator (<>) in Java?

- A) Bitwise operator
- B) Type inference for generics (Java 7+)
- C) Comparison operator
- D) Lambda operator

Answer: B) Type inference for generics (Java 7+)

Explanation: The diamond operator allows type inference: `List<String> list = new ArrayList<>();` instead of repeating the type.

Q79. [Easy]

An immutable object in Java:

- A) Can be modified after creation
- B) Cannot be modified after creation
- C) Is always null
- D) Must be static

Answer: B) Cannot be modified after creation

Explanation: Immutable objects (like String) cannot change state after creation. They are thread-safe by design.

Q80. [Hard]

What is covariant return type in Java?

- A)** Return type must be exactly the same when overriding
- B)** Overriding method can return a subtype of the original return type
- C)** Return type can be any type
- D)** No return type needed

Answer: B) Overriding method can return a subtype of the original return type

Explanation: Since Java 5, an overriding method can return a subclass of the return type declared in the parent method.